

IMAGE-BASED MOTION CONTROL FOR A DRONE

1st Lucia Gumbau Doñate
 Universidad Jaume I
 Castellon, Spain
 al426670@uji.es

Abstract—This paper presents an image-based motion control system that enables operators to navigate a simulated drone using intuitive point-and-click interactions on a live RGB-D camera. Developed using Gazebo, ROS 2, and ArduPilot SITL, the highly modular architecture translates 2D pixel selections into precise 3D spatial commands. By extracting depth map data and applying pitch compensation matrices, the system computes relative distances and executes the movement via the MAVLink protocol. Additionally, it monitors battery telemetry and incorporates an autonomous safe landing zone detection mechanism. Dynamic TF broadcasting is used to render a real-time digital twin in RViz2.

Index Terms—ROS 2, ArduPilot SITL, RGB-D Camera, Gazebo, MAVLink, Digital Twin.

I. OVERVIEW

In this project, a drone operating in simulations is controlled by an image-based interaction system. The system allows the drone to navigate within a 3D world in Gazebo using a simulated RGB-D camera. The drone's movements are determined by converting a user-selected pixel in the input image from the onboard camera into a geometrically 3D motion command relative to the vehicle body frame. The architecture of the project integrates physics simulations, flight control, middleware communication, computer vision processing, and 3D visualization. The complete pipeline was developed using Gazebo, Ardupilot SITL, and ROS2, ensuring modularity, realism, and extensibility.

II. OBJECTIVES

The main objective of this project is the implementation of a vision-driven control system capable of:

- Converting 2D image coordinates into 3D spatial coordinates using depth information and the camera intrinsic projection model.
- Visualizing the drone state and environment in real time.

Some other objectives of the project are:

- Implementing safety logic to critical battery level and landing zone detection
- Structuring the system in a modular and scalable way using ROS 2.
- Enabling real-time monitoring using RViz2.

III. SYSTEM STRUCTURE

The structure of the system is modular and distributed into subsystems, communicating via ROS 2 topics (Figure 1.1):

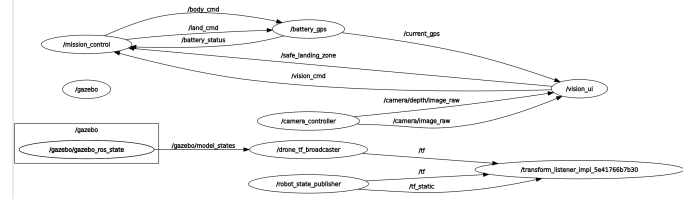


Fig. 1. rqt graph

A. Environment Simulation and Physics

To simulate the 3D world and apply the physics to the rigid bodies in the world, we use Gazebo. This simulator:

- Computes rigid-body dynamics (gravity, inertia, collisions,...).
- Renders camera images from the drone perspective.
- Generates depth maps.
- Publishes model states to ROS 2.

The drone model is defined using a SDF file obtained from GitHub, where the camera and the needed ROS-compatible plugins were added:

- `libgazebo_ros_camera.so`: RGB-D camera interface.
- `libgazebo_ros_state.so`: model state publisher.

B. Flight Control System

ArduPilot SITL (Software In The Loop) is responsible for:

- Performing stabilization and low-level motor control.
- Executing MAVLink commands such as arm, takeoff, land and position control.
- Providing telemetry like battery level and GPS position.

Communication between ROS 2 and ArduPilot is performed using MAVLink over UDP.

C. Middleware and Communication

The system communication is based on ROS 2, which enables topic-based message exchange, real-time telemetry distribution, and transform broadcasting for visualization. The system follows a modular node architecture composed of:

- `battery_gps.py`
- `mission_control.py`
- `vision_ui.py`
- `tf_broadcaster.py`
- `launch.py`

D. Vision-Based Control Module

This subsystem is implemented by the `vision_ui.py` node. It is responsible for acquiring RGB-D data, reconstructing 3D coordinates from image clicks, compensating for camera inclination, safe landing zone detection, and publishing relative motion commands. To obtain the camera data from the simulation, this node subscribes to:

- `/camera/image_raw`
- `/camera/depth/image_raw`
- `/current_gps`

Using `CvBridge`, these ROS image messages are converted into OpenCV matrices. In the depth image, each pixel contains the distance from the camera optical center to the chosen surface, encoded as a 32FC1 matrix. Once it has the matrices, the node performs the 3D reconstruction of the pixel clicked by the user.

- 1) The depth value (Z) is extracted from the depth map.
- 2) The focal lengths are derived from the camera field of view (horizontal and vertical FOV):

$$f_x = \frac{W/2}{\tan(FOV_h/2)} \quad (1)$$

$$f_y = \frac{H/2}{\tan(FOV_v/2)} \quad (2)$$

- 3) With it, the 3D coordinates in the camera optical frame are calculated as:

$$X_{opt} = \frac{(u - c_x) Z}{f_x} \quad (3)$$

$$Y_{opt} = \frac{(v - c_y) Z}{f_y} \quad (4)$$

$$Z_{opt} = Z \quad (5)$$

- 4) Since the camera is 30° downward with respect to the drone body frame, a pitch compensation is applied to obtain the forward and lateral displacements that the drone must perform to reach the selected location:

$$d_{forward} = Z \cos \theta - Y_{opt} \sin \theta \quad (6)$$

$$d_{lateral} = X_{opt} \quad (7)$$

Once it has calculated the displacements, they are published to the topic `vision_cmd` as a `geometry_msgs/Vector3` message that represents a relative displacement in the drone body frame. In addition to the 3D reconstruction of the pixel, this node evaluates the environment for safe landing procedures. The node analyzes the lower region of the depth map to detect flat ground. If the minimum valid depth in this region is greater than 3.0 meters, the terrain is considered safe for descent. Otherwise, if an obstacle is detected below, it computes evasive coordinates to step backward and move rightward to find a clear area. These autonomous coordinates are published to the `/safe_landing_zone` topic.

E. Mission Controller and Safety Layer

This part is controlled by the `mission_control.py`, which is the intermediary between perception and actuation nodes.

On one hand, the node subscribes to the `/vision_cmd` topic, where the vision node publishes the computed displacement and publishes it to the `body_cmd` topic without modifying its values.

On the other hand, this node monitors the battery level via `/battery_status`. If it drops below 20%, or if the user triggers a manual override via a right-click on the camera feed, the system enters an emergency landing mode. During this sequence, the node reads the `/safe_landing_zone` topic. If the drone is perfectly centered over a clear area (within a 1.5m tolerance), it publishes `True` to the `/land_cmd` topic to descend safely. If the area is unsafe, it ignores manual commands and automatically performs a guided search by moving laterally and forward until flat ground is found.

During an emergency landing, if the vision system detects an obstacle below, the drone moves 0.5m backwards and 2m to the right to prevent the drone from colliding with the obstacle and continue scanning for a safe landing zone.

F. MAVLink Interface and Flight Execution

The `battery_gps.py` is the node which interacts with ArduPilot SITL via MAVLink protocols over UDP (127.0.0.1:14550). The initialization procedure of this node is the following:

- 1) MAVLink connection.
- 2) Waits for heartbeat and calibrate sensors.
- 3) Flight mode: GUIDED.
- 4) Arms the vehicle.
- 5) Commands takeoff to 5 meters.

This node subscribes to `body_cmd` topic, where the displacement is specified as a `Vector3` that is translated into a MAVLink `SET_POSITION_TARGET_LOCAL_NED` message. The translation is made using the frame `MAV_FRAME_BODY_OFFSET_NED` with the mask `0b110111111000`. This mask indicates to ArduPilot that the command contains position targets (x,y,z) and zero the yaw rotation, preventing the drone from undesirably spinning. The use of the mask ensures that each click produces a relative motion with respect to the drone's current pose.

Besides, the node also reads the MAVLink messages `BATTERY_STATUS` and `GLOBAL_POSITION_INT`, and publishes this information into the `battery_status` and `current_gps` topics, so the other nodes know the state of the drone.

G. 3D Visualization

A 3D model of the drone is visualized using RViz2 to monitor the drone's behaviour in real time. This layer of the system is composed of:

- **Robot State Publisher:** this node is launched from the simulations launch to read the drone's structural and

geometrical definition from a URDF file, and publishes the transforms between the drone's internal links.

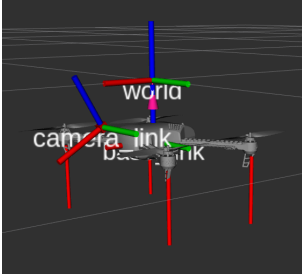


Fig. 2. Drone into RViz2

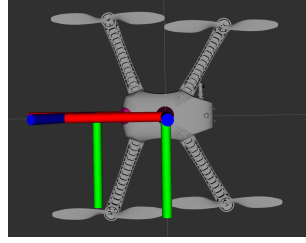


Fig. 3. Drone's plant into RViz2

- `tf_broadcaster.py`: it subscribes to `/gazebo/model_states`, takes the spatial coordinates and quaternion orientation of the drone, and dynamically broadcast this transformations from the absolute world frame to the `base_link` frame.

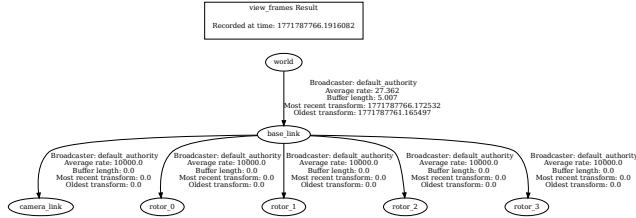


Fig. 4. Transformation Tree

- RViz: it's the graphical interface.

IV. TECHNOLOGIES USED

The system is composed by a depth camera, a computer vision processing libraries (OpenCV) to detect objects and compute their spatial position relative to the robot, and a robotics middleware framework.

A. Depth Camera

This types of cameras generate a per-pixel depth map encoded as a 32FC1 matrix, where each pixel contains the distance from the camera to the observed surface. This allows a direct 3D reconstruction of points in the scene with an accurate distance measurements.

B. ROS 2

ROS 2 facilitates an asynchronous, topic-based communication. Which allows a modular, node-based architecture. The communication with the `vision_ui.py` uses the `qos_profile_sensor_data` (Best Effort) to ensure a smooth video without outdated image frames.

C. OpenCV and CvBridge

OpenCV (Open Source Computer Vision Library) is used to process the matrices from the depth camera. It renderizes the user interface, and capture the user click. However, ROS 2 cannot manipulate image messages, that's why CvBridge is used. CvBridge is a library used to convert ROS sensor messages into NumPy arrays for mathematical processing.

D. ArduPilot SITL and MAVLink

The SILT version of ArduPilot is used to simulate flight dynamics, PID stabilization, and navigation logic. To communicate with ROS 2 MAVLink protocols are used.

E. Gazebo and RViz2

Gazebo simulates the gravity, collisions and realistic sensor rendering. On the other hand, RViz2 is used to visualize the kinematic transform trees (TFs).

V. RESULTS

A. Validation of 3D reconstruction

The system correctly reconstructs the 2D points from the images using the depth information. When the user click on a pixel, the calculated distances show a consistent and accurate behaviour. This coordinates showed consistent behaviour in simulation.

B. Drone response to vision-based commands

Commands published using the `MAV_FRAME_BODY_OFFSET_NED` coordinate system correctly translate into relative motion. The integration of the updated `0b110111111000` MAVLink bitmask mitigated previous issues where the drone would alter its yaw upon reaching target coordinates. The tests results in a smooth and stable movement to the selected point. Showing that the perception, the mission controller and the flight control operate correctly together.

C. Mission control and Safety Behavior

The battery monitoring successfully triggers a forced landing sequence once power drops to 20%. During the landing phase, the drone evaluates the surface below. The tests showed that if an obstacle (such as a building) is detected beneath the drone, it successfully halts its descent, takes a step back, and moves rightward until the camera confirms a safe depth greater than 3.0 meters, ensuring a successful landing.

D. Real-time telemetry

The `tf_broadcast.py` node extracts spatial coordinates from Gazebo correctly, dynamically updating the TF tree spanning from the world origin to the `base_link` and `camera_link`. Allowing the operator to supervise the exact pose and coordinate transforms simultaneously through the RViz2.

E. Overall system performance

The test demonstrates that:

- RGB-D information can be used to guide drone navigation through image interaction.
- The coordinate reconstruction and frame transformations are consistent with the simulation.
- The system supports real-time and safe mission execution.

VI. CONCLUSION

This paper has presented a robust and responsive system using Gazebo, ArduPilot SILENT and ROS 2 to translate a selected 2D pixel of an image from an onboard depth camera into 3D spatial motion commands.

The use of an RGB-D camera overcomes the limitations of a monocular flat-ground assumptions which a traditional camera can not. This approach allows to accurately compute the distances.

The modular node-based architecture makes easier to scale and debug the system. And the combination of the visual perception interface, the mission controller, and the MAVLink flight ensures a reliable, real time performance. Furthermore, the RViz2 integrations with the TF broadcasting provides a digital twin, allowing the operator to monitor everything.

A. Future work

Although the current system perfectly performs the movements indicated by the user and executes autonomous safe landings, future improvements could include:

- The transition of the software architecture from the SITL simulation environment to a physical real-world drone.
- Implementing a forward collision avoidance system to stop the drone if the user clicks behind a physical wall.
- Integrating a Convolutional Neural Network (CNN) to automatically select objects of interest instead of relying on manual point-and-click interactions.
- Improve the autonomous safe landing detection. Currently, the system has a fixed minimum depth threshold. Although it works correctly from a 6m altitude, this static threshold could trigger false obstacle detections if the drone initiates a landing sequence from an altitude lower than 3 meters. Future iterations will dynamically adjust this depth threshold in real-time by fusing it with the drone's current altitude data.

A video demonstration of the system's performance can be viewed at:

https://drive.google.com/file/d/1fikTxaDn_mR3RfDz8BS8_D01Jo8i8QMb/view?usp=drive_link

The source code of the project is available at:

<https://github.com/LuciaGumbau/IR2136.git>